

Experiment 03 · SPI – Synchronous Master-Slave Link

SPI2 in mode 0 at 250 kHz, one command byte per chip-select frame

NUCLEO-F446RE
× 2
STM32F446RET6,
Cortex-M4

SPI2 (APB1)
peripheral used

250 kHz, mode 0
line settings

**4 (NSS, SCK,
MOSI, GND)**
wires between boards

Bare metal
CMSIS registers, no
HAL, no CubeMX

1 Aim and specification

Two NUCLEO-F446RE boards talk to each other over **SPI** and nothing else. The board holding the **master** listing reads its blue B1 user button and measures how long it is held; the board holding the **slave** listing has no button and reacts only to what arrives on the SPI link, blinking its green LD2 in one of two clearly distinguishable patterns. After programming, both boards run from any 5 V USB supply with no PC attached.

1.1 Behavioural specification

Stimulus on the master	Measured press	Byte transmitted	Slave LD2 pattern	LED-to-LED spacing	Total
Short click	< 1000 ms	'S' (0x53)	3 blinks · 150 ms ON / 150 ms OFF	300 ms	0.9 s
Press and hold	≥ 1000 ms	'L' (0x4C)	3 blinks · 1000 ms ON / 1000 ms OFF	2000 ms	6.0 s

The long-press pattern is the point of the experiment: 1000 ms on plus 1000 ms off means the slave LED starts a blink every **2.0 s**, against 300 ms for a short click. Both numbers are the two literals in the slave's `blink()` call, so the timing can be retuned by editing one line. The master flashes its own LD2 for 100 ms each time it sends, so you can always see which board acted.

1.2 How short these programs are

Listing	Lines of code	What it does
Master	about 30	clock enables, four GPIO lines, peripheral init, one loop
Slave	about 25	clock enables, two GPIO lines, peripheral init, one loop

There is no interrupt handler, no clock-tree configuration, no acknowledgement protocol and no checksum. Everything that remains is something an examiner can point at and ask you to justify – which is exactly what you want when you are typing from memory.

2 Apparatus

Qty	Item	Notes
2	NUCLEO-F446RE development board	one typed with the master listing, one with the slave listing
1 set	Male-male jumper wires (Dupont)	keep them under about 20 cm
2	USB Mini-B cables	for programming, then for standalone power
1–2	USB power bank or 5 V phone charger	to run the boards away from the PC
2	4.7 kΩ resistors + small breadboard	I2C experiment only – mandatory bus pull-ups

3 Pin assignment

MCU pin	Function	Configuration	Morpho	Arduino	Used by
PB12	SPI2_NSS	master: output · slave: AF5	CN10-16	–	chip select, active low
PB13	SPI2_SCK	AF5	CN10-30	–	clock, master → slave
PB15	SPI2_MOSI	AF5	CN10-26	–	data, master → slave
PB14	SPI2_MISO	not used	CN10-28	–	the link is one-way
PC13	B1 user button	input + pull-up	CN7-23	–	master only
PA5	LD2 user LED	output	CN10-11	D13	both boards
GND	0 V reference	–	CN10-20	–	mandatory

Reading the headers

CN7 and CN10 are the two 2×19 “morpho” headers along the long edges. Pin 1 of each is the row nearest the ST-LINK part of the board; odd numbers are in the left column and even numbers in the right column when the board is seen from above with the USB connector upwards. Every pin name is printed on the underside of the PCB – when in doubt, find a pin by its *name*, not by counting rows.

4 Circuit and wiring

Make every connection with the boards **unpowered**. The colours in the table are the ones used in Figure 1 on the next page; any colour works electrically, but keeping a convention makes a wiring fault obvious at a glance.

Signal	MASTER pin	SLAVE pin	Wire colour used	Topology
NSS	PB12 · CN10-16	PB12 · CN10-16	violet	straight
SCK	PB13 · CN10-30	PB13 · CN10-30	orange	straight
MOSI	PB15 · CN10-26	PB15 · CN10-26	red	straight
Ground	GND · CN10-20	GND · CN10-20	black	common

Experiment 3 · SPI2 - board-to-board wiring

Top view, USB connectors upwards. Pin names are silk-screened on the underside of each board.

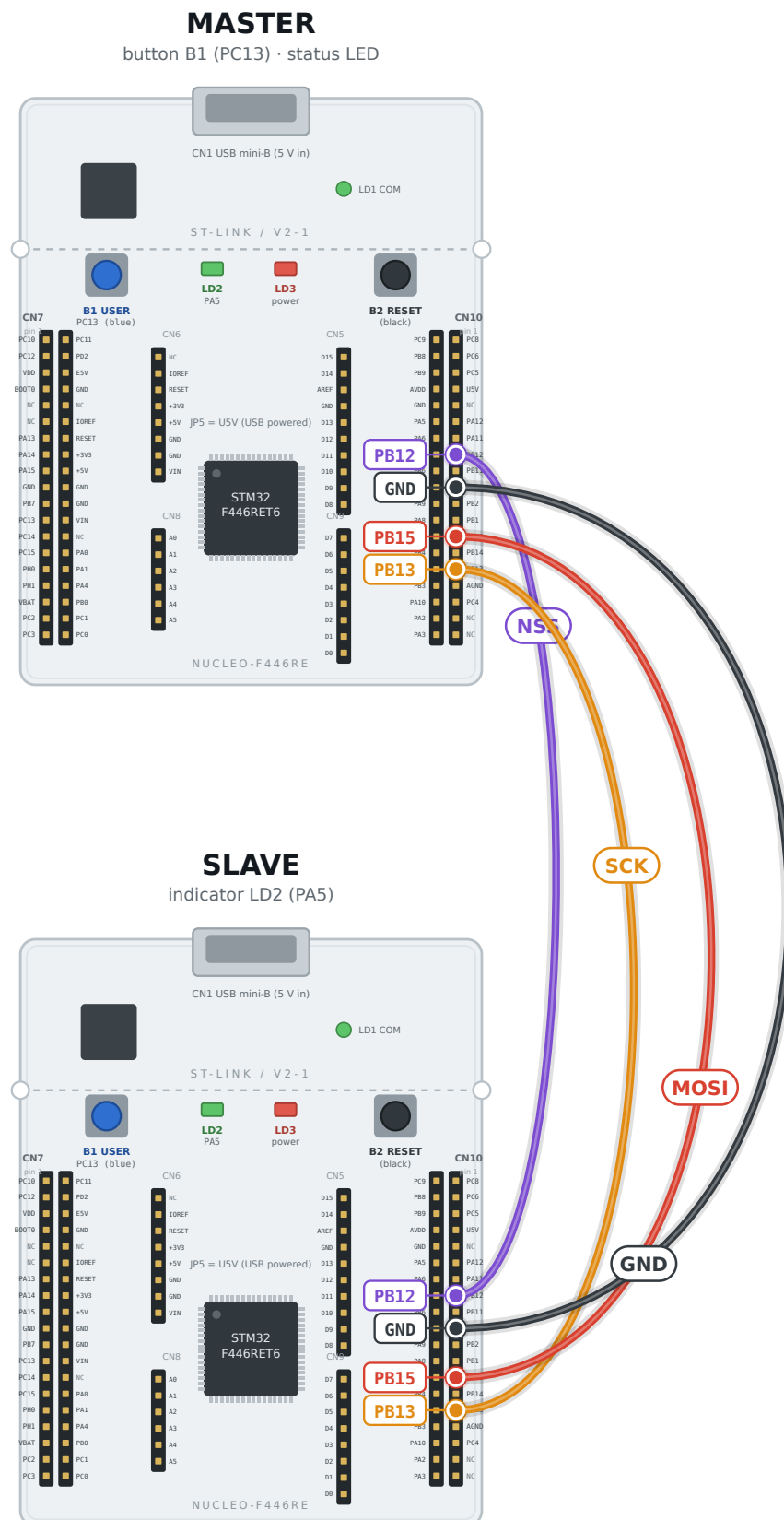


Figure 1 - Physical wiring between the two boards, drawn to the real board outline. The coloured rings mark the pads that carry a wire.

5 Theory of operation

Why SPI2 and not SPI1

`SPI1_SCK` shares pin PA5 with LD2, the green LED this experiment uses as its indicator. Choosing SPI2 (PB12-PB15 on the CN10 morpho header) keeps the LED free. Spotting that kind of pin conflict before wiring is half of interfacing work.

SPI is a **synchronous, single-master** bus built from two shift registers that together form one circular 16-bit register. On every clock edge the master shifts a bit out on MOSI and a bit in on MISO, and the slave does the mirror image. Because the master supplies SCK, there is no baud-rate agreement to negotiate – and the slave has no baud-rate field at all, which is why its whole configuration is the single line `SPI2->CR1 = (1<<6);`.

Mode 0 and the clock speed

Both ends use **mode 0**: `CPOL = 0` (SCK idles low) and `CPHA = 0` (the bit is sampled on the first, rising, edge and changed on the falling edge). On the master the baud field `BR = 101b` divides PCLK1 by 64, giving 16 MHz / 64 = **250 kHz** – deliberately slow, because unshielded jumper wires have no controlled impedance.

Chip select and the BSY flag

The master drives NSS by hand as an ordinary GPIO and sets `SSM = 1` with `SSI = 1` so the peripheral never thinks it lost arbitration and dropped out of master mode. The slave uses **hardware NSS** (`SSM = 0`, PB12 in AF5), so its shift register only moves while the master holds the line low – that is what keeps the two byte counters aligned.

One subtlety worth explaining in the viva: after writing `DR` the master must wait for `BSY` (SR bit 7) to fall, *not* for TXE, before raising NSS. TXE only means the byte has moved from DR into the shift register; raising NSS at that moment would cut the transfer in half.

MISO is left unwired because the slave never answers. Wire PB14-PB14 and the same code would carry a reply byte back in the same clock burst, since SPI is inherently full duplex.

Why there is no clock-configuration code

After reset the STM32F446 runs from its internal 16 MHz RC oscillator (HSI) with every bus prescaler at /1, and CubeIDE's `SystemInit()` does not change that – it only enables the FPU. So **SYSCLK = HCLK = PCLK1 = PCLK2 = 16 MHz** without a single line of setup. That single fact is where the two constants in the listing come from: 16000 in `delay_ms()` (16 MHz / 1 kHz) and the peripheral timing value in the init. Do not paste a `SystemClock_Config()` from a HAL example into these projects. If you raise SYSCLK, the delays and the baud rate move with it.

5.2 The delay function, without an interrupt

All eight listings share the same six-line timer. SysTick is a 24-bit down-counter inside the Cortex-M4 core, so it needs no RCC clock enable and no NVIC setup:

```
SysTick->LOAD = 16000 - 1; /* reload = 1 ms at 16 MHz */
SysTick->VAL = 0;          /* clear the counter */
SysTick->CTRL = 5;         /* bit0 ENABLE, bit2 CLKSOURCE = core clock */
while (ms--) while (!(SysTick->CTRL & (1 << 16))); /* COUNTFLAG */
SysTick->CTRL = 0;
```

COUNTFLAG (bit 16) is set every time the counter reaches zero and is **cleared by reading CTRL**, so the inner loop consumes exactly one millisecond per pass. Because there is no interrupt there is no `SysTick_Handler`, no `volatile` global and no chance of forgetting the handler name – three fewer things to get wrong under exam pressure.

The press length is then measured by counting those milliseconds while the button is held: `while (!(GPIOC->IDR & (1<<13))) { delay_ms(10); t += 10; }`, giving a resolution of 10 ms, which is far finer than the 1000

ms decision threshold.

5.4 Register configuration summary

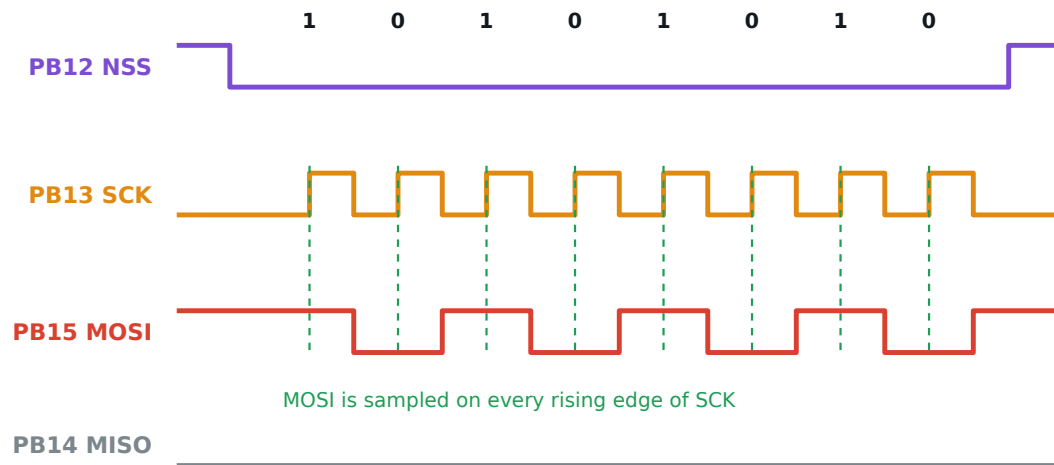
Register	Value written	Why
RCC->APB1ENR	(1<<14)	SPI2 clock – SPI2 and SPI3 are on APB1, SPI1 is on APB2
GPIOB->MODER	(1<<24) master, (2<<24) slave	PB12 is a plain output on the master and an alternate function on the slave
GPIOB->AFR[1]	5 per nibble	AF5 = SPI2
SPI2->CR1 (master)	(1<<2) (5<<3) (1<<9) (1<<8) (1<<6)	MSTR, BR = 101b (PCLK1/64), SSM, SSI, SPE
SPI2->CR1 (slave)	(1<<6)	SPE only – MSTR = 0, mode 0, MSB first, hardware NSS, 8-bit
SPI2->SR	bit 1 TXE, bit 7 BSY	ready for the next byte / a transfer is still in progress
SPI2->SR	bit 0 RXNE	the slave has a complete byte in DR
SPI2->DR	read or write	with DFF = 0 only the low byte is used, so a plain assignment sends one 8-bit frame on the F4 family

5.3 Minimum register sequence - memorise this order

MASTER	SLAVE
1. RCC->AHB1ENR : GPIOA, GPIOB, GPIOC 2. RCC->APB1ENR : SPI2 3. PC13 pull-up, PA5 output 4. PB12 → output, drive it high 5. PB13, PB15 → alternate function, AFRH = 5 6. CR1 = MSTR BR SSM SSI SPE 7. send: NSS low, wait TXE, write DR, wait BSY = 0, NSS high	1. RCC->AHB1ENR : GPIOA, GPIOB 2. RCC->APB1ENR : SPI2 3. PA5 output 4. PB12, PB13, PB15 → alternate function, AFRH = 5 5. CR1 = SPE – that is the whole configuration 6. (no BR, no CPOL/CPHA to change – mode 0 is the reset state) 7. loop: wait RXNE, read DR, blink

6 Signal timing

SPI2 mode 0 (CPOL = 0, CPHA = 0) - first byte of the 4-byte frame



0xAA leaves MSB first while the slave returns don't-care data; the ACK byte 0x5A arrives during the 4th byte.

Figure 2 – SPI mode 0. The master owns SCK, so there is no baud-rate agreement to get wrong; the slave shifts on the edges it is given.

7 MASTER source code `spi_master.c`

Type this over the whole contents of `Core/Src/main.c` in the master project. It is self-contained: no other file to add, no header to edit. The comment block at the top is for you – skip it if you are short of time.

`spi_master.c`

```
1/* EXP 3 SPI2 (mode 0, 250 kHz) --- MASTER --- NUCLE0-F446RE
2 * SPI2 is used because SPI1_SCK is PA5 = LD2.
3 * Wiring (STRAIGHT THROUGH, not crossed):
4 *     PB12 NSS (CN10-16) -> slave PB12 (CN10-16)
5 *     PB13 SCK (CN10-30) -> slave PB13 (CN10-30)
6 *     PB15 MOSI (CN10-26) -> slave PB15 (CN10-26)
7 *     GND      (CN10-20) -> slave GND (CN10-20) */
8
9#include "stm32f446xx.h"
10
11static void delay_ms(uint32_t ms)
12{
13    SysTick->LOAD = 16000 - 1;
14    SysTick->VAL = 0;
15    SysTick->CTRL = 5;
16    while (ms-- > 0) while (!(SysTick->CTRL & (1 << 16)));
17    SysTick->CTRL = 0;
18}
19
20static void spi_send(uint8_t b)
21{
22    GPIOB->ODR &= ~(1 << 12); /* NSS low = select */
23    while (!(SPI2->SR & (1 << 1))); /* TXE */
24    SPI2->DR = b;
25    while (SPI2->SR & (1 << 7)); /* wait BSY = 0 */
26    GPIOB->ODR |= (1 << 12); /* NSS high = release */
27}
28
29int main(void)
30{
31    uint32_t t;
32
33    RCC->AHB1ENR |= (1 << 0) | (1 << 1) | (1 << 2); /* GPIOA, B, C */
34    RCC->APB1ENR |= (1 << 14); /* SPI2 */
35
36    GPIOC->PUPDR |= (1 << 26); /* PC13 pull-up (B1) */
37    GPIOA->MODER |= (1 << 10); /* PA5 output (LD2) */
38    GPIOB->MODER |= (1 << 24); /* PB12 output (NSS) */
39    GPIOB->MODER |= (2 << 26) | (2 << 30); /* PB13, PB15 alternate */
40    GPIOB->AFR[1] |= (5 << 20) | (5 << 28); /* AF5 = SPI2 */
41    GPIOB->ODR |= (1 << 12); /* NSS idles high */
42
43    SPI2->CR1 = (1 << 2) | (5 << 3) | (1 << 9) | (1 << 8) | (1 << 6);
44    /* MSTR BR=/64 SSM SSI SPE */
45
46    while (1) {
47        if (GPIOC->IDR & (1 << 13)) continue;
48
49        t = 0;
50        while (!(GPIOC->IDR & (1 << 13))) { delay_ms(10); t += 10; }
51
52        spi_send((t >= 1000) ? 'L' : 'S');
53
54        GPIOA->ODR |= (1 << 5);
55        delay_ms(100);
56        GPIOA->ODR &= ~(1 << 5);
57    }
58}
```

8 SLAVE source code `spi_slave.c`

Type this over the whole contents of `Core/Src/main.c` in the slave project. It is self-contained: no other file to add, no header to edit. The comment block at the top is for you – skip it if you are short of time.

`spi_slave.c`

```
1/* EXP 3 SPI2 (mode 0) --- SLAVE --- NUCLEO-F446RE
2 * A slave has no baud-rate field: the master's SCK clocks it.
3 * NSS is hardware here (SSM = 0), so PB12 must be wired.
4 * Wiring : PB12 (CN10-16), PB13 (CN10-30), PB15 (CN10-26), GND
5 *         all straight through to the same pins on the master. */
6
7#include "stm32f446xx.h"
8
9static void delay_ms(uint32_t ms)
10{
11    SysTick->LOAD = 16000 - 1;
12    SysTick->VAL = 0;
13    SysTick->CTRL = 5;
14    while (ms-- > 0) while (!(SysTick->CTRL & (1 << 16)));
15    SysTick->CTRL = 0;
16}
17
18static void blink(uint32_t d)
19{
20    for (int i = 0; i < 3; i++) {
21        GPIOA->ODR |= (1 << 5); delay_ms(d);
22        GPIOA->ODR &= ~(1 << 5); delay_ms(d);
23    }
24}
25
26int main(void)
27{
28    RCC->AHB1ENR |= (1 << 0) | (1 << 1); /* GPIOA, GPIOB */
29    RCC->APB1ENR |= (1 << 14); /* SPI2 */
30
31    GPIOA->MODER |= (1 << 10); /* PA5 output (LD2) */
32    GPIOB->MODER |= (2 << 24) | (2 << 26) | (2 << 30); /* PB12,13,15 */
33    GPIOB->AFR[1] |= (5 << 16) | (5 << 20) | (5 << 28); /* AF5 = SPI2 */
34
35    SPI2->CR1 = (1 << 6); /* SPE only: MSTR = 0, */
36                        /* mode 0, MSB, HW NSS */
37    while (1)
38        if (SPI2->SR & (1 << 0)) /* RXNE */
39            blink((uint8_t)SPI2->DR == 'L' ? 1000 : 150);
40}
```


9 Building it in STM32CubeIDE without touching CubeMX

1. **File → New → STM32 Project.** Open the *Board Selector* tab, type `NUCLEO-F446RE`, select it and press *Next*.
2. Name the project, keep *Targeted Language* = **C** and *Targeted Binary Type* = **Executable**, and set **Targeted Project Type = Empty**. This is the step that matters: an *Empty* project gives you the startup file, the linker script, `system_stm32f4xx.c` and the CMSIS register headers, but **no HAL, no LL drivers and no .ioc file**. There is nothing to click in CubeMX because there is no CubeMX.
3. If a dialog offers to initialise all peripherals, answer **No** – with an *Empty* project it normally does not appear at all.
4. Open `Core/Src/main.c`, select everything, delete it, and type the listing from this manual. `#include "stm32f446xx.h"` resolves through `Drivers/CMSIS/Device/ST/STM32F4xx/Include`, which the wizard already placed on the include path. There is no second file to create.
5. Build with **Ctrl + B**, connect the board, then *Run → Run As → STM32 C/C++ Application*.
6. Repeat for the second project with the other listing and flash the second board. Mark the boards **MASTER** and **SLAVE** – once they are off the PC they look identical.

If the build complains

`undefined reference to SystemInit` means the project was created without `system_stm32f4xx.c`; add `void SystemInit(void) { }` at the top of `main.c` – but only in that case, otherwise you get a duplicate symbol. `unknown type name 'uint32_t'` means the device header was not found: check that `STM32F446xx` is listed under *Project → Properties → C/C++ Build → Settings → MCU GCC Compiler → Preprocessor*.

10 Running the pair with no PC attached

Nothing in these programs needs the debugger. As soon as the board is powered, the reset vector runs `main()` from flash, so the pair works on its own.

Option A – one USB supply per board (recommended)

1. Unplug both boards from the PC after flashing.
2. Plug each board's CN1 Mini-B socket into a USB power bank, a 5 V phone charger or a powered hub. Leave jumper **JP5 on U5V**, its factory position.
3. LD3 (red) lights on each board. Keep the protocol wires and the common ground exactly as they were during programming.

Option B – one 5 V source for both boards

1. Move **JP5 to E5V** on *both* boards.
2. Feed +5 V into **E5V** (CN7 pin 6) and 0 V into **GND** (CN7 pin 8 or CN10 pin 20) on both boards from the same supply.
3. The grounds are then already common, so the separate GND jumper becomes redundant – leaving it in place does no harm.

Safety

Never drive 5 V into the **+5V** pin while JP5 is on U5V *and* a USB cable is plugged in – that connects two supplies together. Pick one scheme and stay with it. BOOT0 must stay low (its default) so the MCU boots from flash.

11 Test procedure and observations

1. Flash the master listing to board 1 and the slave listing to board 2.
2. Power both boards down, then wire them as in the connection table. Check the ground wire twice – a missing ground is the single most common cause of a dead link.
3. Power both boards. LD3 (red) must be lit on each.
4. **Test 1 - short click.** Press and release B1 on the master in well under a second. The master's LD2 gives one short flash and the slave answers with three fast blinks.
5. **Test 2 - long press.** Hold B1 for at least one second, then release. The slave now blinks three times with one second on and one second off – two seconds from the start of one blink to the start of the next.
6. Time the slow pattern with a stopwatch. Repeat each test three times and record it.
7. **Test 3 - boundary.** Try a press of about one second and confirm which pattern you get. The threshold is the `t >= 1000` comparison in the master.
8. If a scope or logic analyser is available, capture the line(s) during one press and compare with the timing figure in section 6.

Trial	Press duration (ms)	Byte expected	Master LD2 flashed	Blinks counted	Measured spacing (s)	Pass
1						
2						
3						
4						
5						
6						

12 Troubleshooting

Symptom	Cause and remedy
The slave receives nothing	Unlike UART, SPI is wired straight through – SCK to SCK, MOSI to MOSI. Also check that the NSS wire is in place: with hardware NSS the slave shifts nothing while PB12 is high.
Received bytes are shifted by one bit or occasionally wrong	The master raised NSS before BSY cleared, or CPOL/CPHA differ between the two boards. Both must be mode 0.
It works at 250 kHz but fails when BR is reduced	Expected with long unshielded jumpers. Shorten the wires before raising the clock.
The slave never reacts	Ground is not common. Join GND (CN10-20) of the two boards – this is by far the most frequent fault.
The master's LD2 does not flash either	The master board is not running, or B1 was not detected. Check LD3 (red) is lit and that you typed <code>GPIOC->PUPDR = (1 << 26);</code> – without the pull-up, PC13 floats and never reads as pressed.
Blink timing is wrong (much faster or slower than 2 s)	SYSCLK is not 16 MHz. Make sure no <code>SystemClock_Config()</code> or PLL setup was left in <code>main.c</code> , and that <code>SysTick->LOAD</code> is 16000 – 1.
Both boards blink but the roles are swapped	The master listing was flashed to the slave board. Label the boards.
A long press is read as a short one	You released before 1000 ms. The threshold is the literal <code>1000</code> in the master's <code>t >= 1000</code> test – change it there if your lab asks for a different value.

13 Review questions

1. Why does SPI need no baud-rate agreement while UART does?
2. Explain CPOL and CPHA, and list the four SPI modes.
3. What is NSS for, and what is the difference between $SSM = 0$ and $SSM = 1$?
4. Why must the master poll BSY rather than TXE before releasing NSS?
5. Why was SPI2 chosen instead of SPI1 on this board?
6. Why can a byte never be received on SPI without one being sent?
7. How many slaves can share one SPI bus, and what limits the number?
8. Compare the pin count and maximum speed of SPI, I2C and UART.